LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL 6 – Inheritance



NAMA: Lintang Anissa Hakim

NIM: 103132400040

KELAS: S1-SD-05

PROGRAM STUDI SAINS DATA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

Inheritance (pewarisan) adalah salah satu pilar OOP yang memungkinkan sebuah kelas diturunkan dari kelas lain, sehingga kelas tersebut otomatis memiliki atribut dan method dari kelas asalnya. Hubungannya disebut *"is a relationship"*. Tujuannya menghindari duplikasi kode, karena field dan method yang sama cukup ditulis sekali di kelas induk. Semakin ke bawah dalam hierarki, kelas menjadi semakin spesifik. Kelas yang menurunkan disebut **superclass** (base class), sedangkan kelas yang diturunkan disebut **subclass** (derived class). Di Java, pewarisan ditulis dengan keyword extends:
class B extends A { ... }

Keyword super dipakai subclass untuk merujuk ke superclass, yaitu untuk memanggil constructor (super(nama);), mengakses atribut (super.departemen), dan memanggil method(super.tampilNama();). Pemanggilan super(...) pada constructor harus berada di baris pertama.

Overloading method adalah pembuatan dua atau lebih method dengan nama yang sama dalam satu kelas, tetapi jumlah atau tipe parameternya berbeda. Contohnya IsiParam(String) dan IsiParam(String, String).

Overriding method terjadi ketika subclass mendefinisikan ulang method milik superclass dengan nama dan parameter yang sama. Jika method dipanggil dari objek subclass, yang dijalankan adalah versi milik subclass. Versi superclass tetap bisa dipanggil lewat super.namaMethod().

Polymorphism adalah kemampuan sebuah objek mengambil beberapa bentuk. Satu variabel bertipe superclass bisa menampung objek dari berbagai subclass, dan method yang dijalankan menyesuaikan jenis objek aslinya.

B. Guided 1

[A]

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to
change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this
template
 */
package Guided;

/**
 *
 * @author QWERTY
 */
public class A {
   int x;
    int y;
```

```
    void tampilXY() {
        System.out.println("Nilai x: " + x + " nilai y: " + y);
    }
}
```

[BextendsA]

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
 */
package Guided;

/**
 *
 * @author QWERTY
 */
public class BextendsA extends A {
  int z;
    int hasil;

    void jumlahXYZ() {
        hasil = x+y+z;

        System.out.println("jumlah (x + y + z) = " + hasil);
    }
}
```

[Demo]

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
 */
package Guided;

/**
 *
 * @author QWERTY
 */
public class Demo {
    public static void main(String[] args) {
      A superclass = new A();
      BextendsA subclass = new BextendsA();

        superclass.x = 3;
        superclass.y = 4;
```

```
                System.out.println("Superclass: ");
                superclass.tampilXY();


                // member superclass dapat diakses dari subclassnya
                subclass.x = 1;
                subclass.y = 2;
                subclass.z = 5;

                System.out.println("Subclass: ");
                subclass.jumlahXYZ();

            }
}
```

Output :

```
Superclass:
Nilai x: 3 nilai y: 4
Subclass:
jumlah (x + y + z) = 8
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
Total time:  5.122 s
Finished at: 2026-10-26T20:41:46+07:00
------------------------------------------------------------------------
```

Guided 2

[Manager]

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
 */
package Guided2;

/**
 *
 * @author QWERTY
 */
public class Manager extends Employee  {
    private String alamat;

    public Manager(String nama, String alamat) {
        /* memanggil konstruktor employee */
```

```
        super(nama);
        this.alamat = alamat;
    }

    public void tampilInformasi() {
        super.tampilNama();

        /* menampilkan variabel alamat dan departemen */
        System.out.println("alamat : " + alamat);
        System.out.println("departemen : " + super.departemen);
    }
}
```

[Employee]

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change
this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this
template
 */
package Guided2;

/**
 *
 * @author QWERTY
 */
public class Employee {
     private String name;
    String departemen;

    public Employee(String nama) {
        this.name = nama;
    }

    public void tampilNama() {
        System.out.println("nama : " + name);
    }
}
```

[Demo Manager]

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change
this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this
template
 */
package Guided2;
```

```
/**
 *
 * @author QWERTY
 */
public class DemoManager {
    public static void main(String[] args) {
      /* membuat objek */
      Manager adi = new Manager("Adi", "Sukabirus");
      adi.departemen = "SDM";
      adi.tampilInformasi();
   }
}
```

Output :

```
  nama : Adi
  alamat : Sukabirus
- departemen : SDM
  ------------------------------------------
  BUILD SUCCESS
  ------------------------------------------
  Total time:  7.278 s
  Finished at: 2026-10-26T20:53:44+07:00
  ------------------------------------------
```

Guided 3

[Lagu]

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
 */
package Guided3;

/**
 *
 * @author QWERTY
 */
public class Lagu {
   String pencipta;
   String judul;

   void IsiParam(String param1) {
      judul = param1;
      pencipta = "Tidak dikenal";
   }
```

```
    void IsiParam(String param1, String param2) {
        judul = param1;
        pencipta = param2;
    }

    void Tampilkan() {
        System.out.println("Judul : " + judul + ", pencipta : " + pencipta);
    }
}
```

[Main]

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
 */
package Guided3;

/**
 *
 * @author QWERTY
 */
public class Main {
    public static void main(String[] args) {
        Lagu d, e;
        d = new Lagu();
        e = new Lagu();

        d.IsiParam("Lagu 1");
        e.IsiParam("Lagu 2", "aku");

        d.Tampilkan();
        e.Tampilkan();
    }
}
```

Output :

```
--- exec:3.6.3:exec (default-cli) @ Modul6
Judul : Lagu 1, pencipta : Tidak dikenal
Judul : Lagu 2, pencipta : aku
------------------------------------------------
BUILD SUCCESS
------------------------------------------------
Total time:  7.247 s
Finished at: 2026-10-26T20:58:37+07:00
------------------------------------------------
```

Guided 4

[A]

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
 */
package Guided4;

/**
 *
 * @author QWERTY
 */
public class A {
    public void tampilkan() {
        System.out.println("Method milik class A dipanggil...");
    }
}
```

[B]

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
 */
package Guided4;

/**
 *
 * @author QWERTY
 */
public class B extends A {
    public void tampilkan() {
        super.tampilkan(); // Method milik superclas dipanggil
        System.out.println("Method milik class B dipanggil...");
    }
}
```

[Main]

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
```

```
 */
package Guided4;

/**
 *
 * @author QWERTY
 */
public class Main {
    public static void main(String[] args) {
       B overidB = new B();
       overidB.tampilkan();
    }
}
```

Output :

```
--- exec:3.6.3:exec (default-cli) @ Modul6 ·
Method milik class A dipanggil...
Method milik class B dipanggil...
------------------------------------------·
BUILD SUCCESS
------------------------------------------·
Total time:  5.961 s
Finished at: 2026-10-26T21:05:33+07:00
------------------------------------------·
```

C. Unguided

1. Soal Unguided 1

Buatlah program Java untuk membandingkan estimasi biaya komputasi tiga jenis model *machine learning* (semua atribut private):

a. **Kelas induk** (nama model, jumlah data latih) dengan method *overloading* hitung biaya: tanpa parameter (data × 2) dan dengan parameter epoch (data × epoch).
b. **Model regresi** (turunan induk, tambah jumlah fitur): *override* hitung biaya = biaya induk + (fitur × 10), plus method baru dengan parameter epoch dan batch = data × epoch ÷ batch.
c. **Model klasifikasi** (turunan induk, tambah jumlah kelas): *override* hitung biaya = biaya induk × kelas.
d. **Model deep learning** (turunan model klasifikasi, tambah jumlah layer): *override* hitung biaya = biaya kelas di atasnya + (layer × 200).
e. **Main**: buat objek Regresi Linier (500, 4 fitur), Decision Tree (800, 3 kelas), dan Neural Network (1000, 5 kelas, 3 layer) dalam satu array bertipe kelas induk. Tampilkan info dan biaya tiap model, total biaya, serta biaya 10 epoch (Neural Network dan Regresi Linier) dan 10 epoch batch 50 (Regresi Linier).

Model Regresi

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
 */
package Unguided;

/**
 *
 * @author QWERTY
 */
public class ModelRegresi extends ModelML {
    private int jumlahFitur;

    public ModelRegresi(String nama, int jumlahDataLatih, int jumlahFitur) {
        super(nama, jumlahDataLatih);
        this.jumlahFitur = jumlahFitur;
    }

    @Override
    public void tampilkanInfo() {
        super.tampilkanInfo();
        System.out.println("Jumlah fitur: " + jumlahFitur);
    }

    @Override
    public int hitungBiaya() {
        return super.hitungBiaya() + (jumlahFitur * 10);
    }

    public int hitungBiaya(int epoch, int ukuranBatch) {
        return getJumlahDataLatih() * epoch / ukuranBatch;
    }
}
```

Model ML

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
 */
package Unguided;

/**
 *
 * @author QWERTY
 */
```

```
public class ModelML {
    private String nama;
    private int jumlahDataLatih;

    public ModelML(String nama, int jumlahDataLatih) {
        this.nama = nama;
        this.jumlahDataLatih = jumlahDataLatih;
    }

    public int getJumlahDataLatih() {
        return jumlahDataLatih;
    }

    public void tampilkanInfo() {
        System.out.println("Model: " + nama + " | Data latih: " + jumlahDataLatih);
    }

    public int hitungBiaya() {
        return jumlahDataLatih * 2;
    }

    public int hitungBiaya(int epoch) {
        return jumlahDataLatih * epoch;
    }
}
```

Model Klasifikasi

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to
change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this
template
 */
package Unguided;

/**
 *
 * @author QWERTY
 */
public class ModelKlasifikasi extends ModelML {
    private int jumlahKelas;

    public ModelKlasifikasi(String nama, int jumlahDataLatih, int jumlahKelas) {
        super(nama, jumlahDataLatih);
        this.jumlahKelas = jumlahKelas;
    }

    @Override
    public void tampilkanInfo() {
        super.tampilkanInfo();
```

```
        System.out.println("Jumlah kelas: " + jumlahKelas);
    }

    @Override
    public int hitungBiaya() {
        return super.hitungBiaya() * jumlahKelas;
    }
}
```

Model Deep Learning

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
 */
package Unguided;

/**
 *
 * @author QWERTY
 */
public class ModelDeepLearning extends ModelKlasifikasi {
    private int jumlahLayer;

    public ModelDeepLearning(String nama, int jumlahDataLatih, int jumlahKelas, int jumlahLayer) {
        super(nama, jumlahDataLatih, jumlahKelas);
        this.jumlahLayer = jumlahLayer;
    }

    @Override
    public void tampilkanInfo() {
        super.tampilkanInfo();
        System.out.println("Jumlah layer: " + jumlahLayer);
    }

    @Override
    public int hitungBiaya() {
        return super.hitungBiaya() + (jumlahLayer * 200);
    }
}
```

Main

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
```

```
 */
package Unguided;

/**
 *
 * @author QWERTY
 */
public class Main {
    public static void main(String[] args) {
        ModelML[] daftarModel = {
            new ModelRegresi("Regresi Linier", 500, 4),
            new ModelKlasifikasi("Decision Tree", 800, 3),
            new ModelDeepLearning("Neural Network", 1000, 5, 3)
        };

        int totalBiaya = 0;

        for (ModelML m : daftarModel) {
            m.tampilkanInfo();
            int biaya = m.hitungBiaya();
            System.out.println("Biaya komputasi: " + biaya);
            System.out.println("---");
            totalBiaya += biaya;
        }

        System.out.println("Total biaya: " + totalBiaya);
        System.out.println("Biaya 10 epoch (Neural Network): " +
daftarModel[2].hitungBiaya(10));
        System.out.println("Biaya 10 epoch (Regresi Linier): " +
daftarModel[0].hitungBiaya(10));

        ModelRegresi regresi = (ModelRegresi) daftarModel[0];
        System.out.println("Biaya 10 epoch batch 50 (Regresi Linier): " +
regresi.hitungBiaya(10, 50));
    }
}
```

Output :

```
--- exec:3.6.3:exec (default-cli) @ Modul6 ---
Model: Regresi Linier | Data latih: 500
Jumlah fitur: 4
Biaya komputasi: 1040
---
Model: Decision Tree | Data latih: 800
Jumlah kelas: 3
Biaya komputasi: 4800
---
Model: Neural Network | Data latih: 1000
Jumlah kelas: 5
Jumlah layer: 3
Biaya komputasi: 10600
---
Total biaya: 16440
Biaya 10 epoch (Neural Network): 10000
Biaya 10 epoch (Regresi Linier): 5000
Biaya 10 epoch batch 50 (Regresi Linier): 100
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
Total time:  3.199 s
Finished at: 2026-10-26T21:07:33+07:00
------------------------------------------------------------------------
```

Penjelasan :

Program Unguided ini membandingkan estimasi biaya komputasi dari tiga jenis model *machine learning* dengan memanfaatkan konsep inheritance. Kelas induk ModelML menyimpan nama model dan jumlah data latih, serta memiliki dua method hitungBiaya dengan nama yang sama tetapi parameter berbeda (*overloading*). Dari kelas ini diturunkan ModelRegresi yang menambah atribut jumlah fitur, dan ModelKlasifikasi yang menambah atribut jumlah kelas. Kemudian ModelDeepLearning diturunkan dari ModelKlasifikasi dengan tambahan atribut jumlah layer, sehingga terbentuk pewarisan bertingkat. Setiap kelas turunan meng-*override* method tampilkanInfo() dan hitungBiaya() dengan memanggil versi kelas di atasnya lewat super, lalu menambahkan perhitungan sesuai atribut miliknya. ModelRegresi juga memiliki method hitungBiaya tambahan dengan parameter epoch dan ukuran batch.

Pada class Main, ketiga objek disimpan dalam satu array bertipe ModelML, yang dimungkinkan oleh konsep *polymorphism*. Saat perulangan memanggil hitungBiaya(), Java otomatis menjalankan versi sesuai jenis objek aslinya, sehingga diperoleh biaya Regresi Linier 1040, Decision Tree 4800, dan Neural Network 10600, dengan total 16440. Untuk memanggil hitungBiaya(10, 50), elemen array harus di-*cast* ke ModelRegresi karena method tersebut hanya dimiliki kelas itu.

D. Kesimpulan

Dari praktikum Modul 6 dapat disimpulkan bahwa inheritance memungkinkan sebuah kelas mewarisi atribut dan method dari kelas lain menggunakan keyword extends, sehingga duplikasi kode dapat dihindari dan program menjadi lebih terstruktur. Keyword super digunakan subclass untuk memanggil constructor, atribut, maupun method milik superclass. Praktikum ini juga menunjukkan perbedaan *overloading*, yaitu method bernama sama dengan parameter berbeda dalam satu kelas, dan *overriding*, yaitu subclass mendefinisikan ulang method milik superclass. Melalui tugas Unguided, konsep tersebut diterapkan pada pewarisan bertingkat (ModelML → ModelKlasifikasi → ModelDeepLearning) serta polymorphism, di mana objek berbagai subclass dapat disimpan dalam satu array bertipe superclass dan method yang dijalankan menyesuaikan jenis objek aslinya. Hasilnya, program berhasil menghitung biaya komputasi tiap model dengan total 16440 sesuai contoh output pada soal.